Math reasoning and formulas

Mexico finals team reference

Built from the 2017-2025 regional research and pol-cova/kactl. A companion to the existing KACTL notebook, with explanations, contracts, and tested C++17 helpers.

Use the recognition tables first. Check assumptions before typing. All helper indices are zero-based unless explicitly stated. Advanced KACTL entries are navigation references; consult their source and listed dependencies. The team must confirm event material limits before choosing what to print.

Edition 1 - 18 October 2026
Editable sources and tests: handbooks/ in the repository.
This document does not replace the original kactl.pdf.

Contents and code lookup

Math at the contest table **3**

Find the right question first . . . 3
Exact integer arithmetic . . . 3
Divisibility, prime factors and value-domain counting . . . 4
Congruences and cyclic states . . . 5
Worked regional reductions . . . 6
Counting without enumerating every object . . . 7
Probability and expectations . . . 7
Invariants, inequalities and choosing a proof . . . 8
Integer geometry essentials . . . 8
Scope and evidence . . . 9
Copyable C++17 helpers . . . 9
Code conventions and includes . . . 9
Checked narrowing and signed division . . . 10
Exact decimals and digit counts . . . 10
Modular arithmetic and binary powers . . . 10
Extended Euclid, inverses and CRT . . . 11
Trial factorization, totients and sieve . . . 11
Multiplicative order and decimal shape . . . 12
Combinations under a prime modulus . . . 12
Strict inversions by compressed Fenwick tree . . . 12
Fuzzy factorization construction . . . 12
Exact integer geometry . . . 13

Math at the contest table

This companion explains the math decisions behind the code. Use the implementation handbook for graph and DP mechanics. All examples here use integers unless a section explicitly introduces probability or real geometry. The helpers at the end are in namespace `finals_math`; copy only the module and dependencies you need.

The selection follows our 2017-2025 regional research and the team's difficulties with Fuzzy Factorization, decimal representations, and turning brute force into counting. This is a broad reference, not a prediction of the next set. Topics marked advanced are available in the original KACTL when a teammate understands the reduction and constraints.

Find the right question first

Statement clue	First calculation	Tool to look up	Main trap
Round up batches, signed intervals	Derive floor or ceiling with an example below zero	`floor_div`, `ceil_div`	C++ truncates toward zero
Prices, exact decimal input	Count fixed fractional places	`parse_fixed`	Binary floating point changes equality
Enormous decimal integer, allowed error	Bound the effect of dropping digits	Fuzzy construction	Factoring the original integer
Divisibility of a decimal prefix	Retain its remainder	Modular states	Carrying an unnecessary huge value
Several repeating schedules	Write congruences and check compatibility	`crt`	Moduli need not be coprime
Many divisor queries with bounded values	Check maximum value and query count	`Sieve`, multiples counting	Trial division per query
Rational decimal length	Reduce denominator, strip 2 and 5	`decimal_shape`	Period need not equal phi
Choose groups or arrange roles	Decide ordered/unordered and repeated/distinct	Combinations	Factorial division modulo a composite
Sum over all subarrays or pairs	Count each element's contribution	Contributions, inversions	Multiplying by the wrong multiplicity
Expected answer	Define uniform outcomes; use indicator variables	Expectation	Assuming independence for linearity
Allowed operations, winning states	Find a quantity unchanged by an operation	Invariants	Examples alone are not a proof
Left/right, collinear, polygon area	Subtract coordinates, take cross products	`orient`, dot, area	Overflow before widening

Exact integer arithmetic

Floor, ceiling, remainder, and interval counts

For positive b, mathematical division gives a = b*q+r with 0 <= r < b and q=floor(a/b). C++ uses truncation toward zero, so `-7/3 == -2` and `-7%3 == -1`. The mathematical floor is -3. Our helpers correct the sign using a 128-bit intermediate, and support either sign of b. Both reject b=0 and throw if the mathematical quotient exceeds signed 64-bit range.

Examples: floor(-7/3)=-3, ceil(-7/3)=-2; floor(7/-3)=-3, ceil(7/-3)=-2. For nonnegative a and positive b, ceiling is also `a/b + (a%b != 0)`. Avoid `(a+b-1)/b` when the addition might overflow.

The multiples of positive d in inclusive [L,R] number `floor(R/d)-ceil(L/d)+1` if the first index does not exceed the last. Perform the subtraction in a wide type. Example [-7,7], d=3 gives multiples -6,-3,0,3,6: five. An integer interval with strict endpoint x<U becomes x<=U-1 only if U-1 fits the type.

Time is O(1), space O(1). Never use a rounding helper without checking whether the problem allows negative values. For normalized residues, `norm(a,m)` returns a value in [0,m) and requires positive m.

Overflow is part of the proof

Signed 64-bit fits about 9.22e18. Signed overflow is undefined behavior in C++. `long long product = a*b` does not help when a and b are `int`: multiplication already happened in 32 bits. Write `i128(a)*b` before arithmetic. Widen coordinate subtraction too: `i128(b.x)-a.x`.

Useful estimates: n(n-1)/2 is about n squared /2; sums of n values of magnitude A need about n*A; a sum of all subarray sums can need about n cubed *A. Bounding a single transition does not bound the accumulated answer.

`i128` is the GCC/Clang `__int128_t` extension, not ISO C++17. It is supported by the tested compiler. `narrow` checks the final conversion to `long long`. No helper can rescue an earlier overflow in the caller. For 128-bit output, repeatedly divide a nonnegative magnitude by ten; handle the sign separately. These helpers do not provide arbitrary-precision integers.

Exact decimal parsing and digit counts

When input has at most k fractional digits, store the value scaled by 10^k. With k=2, `12.05` becomes 1205, `3` becomes 300, and `-0.07` becomes -7. Then addition, equality and remainder tests are exact. `parse_fixed` rejects too many fractional digits instead of silently rounding; `12.005` is invalid for cents. The parser accepts a sign and an integer part, with optional decimal point. It does not accept exponent notation, commas or leading whitespace.

Read the input as text. Multiplying an already-read `double` by 100 can be wrong before any integer conversion. If the statement specifies rounding, derive that rounding separately from parsing.

`digits10` counts magnitude digits; zero has one digit and a minus sign is excluded. Repeated integer division costs O(number of digits) and handles powers of ten exactly. A logarithm can fall on the wrong side of a power-of-ten boundary. Counting the integer digits of a/b uses `digits10(a/b)` for a>=0,b>0; it does not count fractional digits.

Divisibility, prime factors and value-domain counting

GCD and Bezout coefficients

The common divisors of a and b are exactly those of b and a%b. Euclid repeatedly applies this identity in O(log min(a,b)) divisions. For nonnegative inputs, `std::gcd(a,b)` suffices. Avoid `std::gcd(LLONG_MIN,...)` because its positive magnitude cannot be represented as a signed 64-bit value.

Extended Euclid returns g,x,y satisfying a*x+b*y=g. Our helper requires nonnegative a,b and returns coefficients in 128 bits. It is useful when solving a*x+b*y=c: solutions exist exactly when g divides c. Multiply one coefficient solution by c/g. Every solution has x=x0+t*(b/g), y=y0-t*(a/g). Intersect constraints on t with floor/ceiling division; the helper does not perform that interval intersection.

Example: 6x+9y=3 has x=-1,y=1. For right side 4, gcd=3 does not divide 4, so there is no integer solution. When both coefficients are zero, handle c=0 (all pairs) and c!=0 (none) separately.

Original KACTL reference: `content/number-theory/euclid.h`. Our interface has explicit domains and wider coefficients; it is not a drop-in replacement.

Factorization, divisor formulas and totients

For n=product p^e, each divisor independently selects an exponent from 0..e. Thus:

- divisor count tau(n)=product(e+1);
- divisor sum sigma(n)=product(1+p+...+p^e);
- Euler totient phi(n)=n*product((p-1)/p), over distinct prime factors.

For 72=2^3*3^2, tau=4*3=12, sigma=(1+2+4+8)*(1+3+9)=195, phi=72/2*1/3*2=24. The empty factorization of 1 gives tau(1)=sigma(1)=phi(1)=1.

`factor` divides by 2 then odd candidates while p<=remaining/p. A remaining value greater than one is prime: otherwise a factor at most its square root would have been removed. Worst-case O(sqrt(n)) divisions, O(log n) factor entries. Factoring 1e10 needs candidates only through 1e5. Repeating trial division on 1e18 primes is unsuitable. For unrestricted 64-bit factorization consult KACTL `MillerRabin.h`, `Factor.h`, and their stress tests; they are advanced modules with extra dependencies.

Compute divisor sums as a short geometric accumulation with checked wide arithmetic. The closed form `(p^(e+1)-1)/(p-1)` is an integer identity but division modulo a modulus is unsafe unless p-1 is invertible. A recurrence `term*=p; sum+=term` avoids modular division entirely. Generating divisors means expanding a list with all old divisors times p,p^2,...,p^e; there are tau(n) outputs, so account for that cost.

`Sieve(V)` stores a smallest prime factor for each 2..V. The linear sieve takes O(V) time and O(V) memory; factoring x<=V then takes O(number of prime factors with multiplicity), at most O(log x). `spf[0]` and `spf[1]` are not prime factors. Keep V

feasible in memory and nonnegative.

Histogram, multiples and exact GCD classes

When there are N occurrences but values only in 1..V, a frequency table often beats a pair loop. Let freq[x] count occurrences. Then `mult[d]=sum(freq[k*d])` counts values divisible by d. Computing all mult takes sum floor(V/d)=O(V log V), memory O(V). This assumes positive values: zero is divisible by every positive d and needs separate handling.

For fixed denominator b, count numerators whose gcd(a,b) is exactly g, for each g dividing b. Process divisors in descending order:

`exact[g] = mult[g] - sum(exact[h] for h|b, h>g, g|h)`.

Why: every value divisible by g falls into exactly one gcd class h divisible by g. Example b=6, values [2,3,6,6]: exact[6]=2, exact[3]=1, exact[2]=1, exact[1]=0. Repeated occurrences matter. Weight a denominator class by freq[b] too when counting ordered occurrence pairs.

The nested-divisor work is not O(V log V) automatically. Estimate sum tau(b)^2 or derive a better ordering before promising a time bound. Our Displaying Decimals research validated the length formula and small cases; it did not benchmark maximum-size aggregation.

Congruences and cyclic states

Modular multiplication, powers and inverses

Congruence a=b modulo m means m divides a-b. Addition, subtraction and multiplication preserve it. Division requires an inverse: b*x=1 modulo m has a solution exactly when gcd(b,m)=1. `inverse` returns `nullopt` when no inverse exists, requires m>1, and uses extended Euclid. Example inverse(3,8)=3, but inverse(2,8) does not exist.

`mul_mod` uses a signed 128-bit product of normalized nonnegative 64-bit residues; positive m<=LLONG_MAX fits this product. `pow_mod(a,e,m)` uses binary exponentiation in O(log e) modular multiplications and O(1) space, with unsigned nonnegative exponent. `pow_mod(a,0,m)=1%m`, including a=0. Decide whether 0^0=1 matches the mathematical counting model.

If p is prime and a is not divisible by p, Fermat gives a^(p-1)=1 modulo p and inverse(a)=a^(p-2). This shortcut is false for general composite moduli. Even exponent reduction modulo p-1 needs care when a=0 modulo p and the true exponent may be positive.

Original references: `content/number-theory/ModPow.h`, `ModInverse.h`, `ModularArithmetic.h`. Our `mul_mod` avoids overflow that direct `a*b%m` can have.

Chinese remainder theorem with shared factors

To merge x=a modulo m and x=b modulo n, let g=gcd(m,n). A solution exists exactly when g divides b-a. Write x=a+m*t; solve `(m/g)*t=(b-a)/g` modulo n/g. Extended Euclid supplies the inverse. The combined modulus is lcm(m,n)=m*(n/g), and the answer is unique modulo this lcm.

Example x=2 modulo 6 and x=5 modulo 9 gives x=14 modulo 18. Changing the second residue to 4 makes the system incompatible, because 4-2 is not divisible by 3. Normalize negative residues first. Merge several equations one by one, stopping on incompatibility.

`crt` accepts positive signed 64-bit moduli, returns the smallest nonnegative residue, and throws if the lcm cannot fit signed 64 bits. Each merge takes O(log min(m,n)); a mathematically valid solution can still exceed our representation. Modulus 1 is allowed and adds no restriction. See original `content/number-theory/CRT.h` for its different interface and assumptions.

Decimal prefixes and modular DP

If a decimal prefix has remainder r modulo m, appending digit d gives `(10*r+d)%m`. It follows directly from new_value=10*old_value+d. Widen the multiplication if m can be large. Leading-zero restrictions belong to the state or allowed transitions.

Two prefixes at the same position with the same remainder have the same possible suffix completions when no other restriction distinguishes them. That is the proof for merging them into a DP state. For a digit pattern with '?' entries, define possible[pos][r] or count[pos][r], set the empty prefix remainder to zero, and transition through the allowed digits. Naive time O(length*m*10), memory O(m) for counts or O(length*m) when reconstructing with stored choices.

Existence, count and smallest reconstruction are separate tasks. For the smallest number, build suffix feasibility and greedily try digits in increasing order while preserving a feasible completion. A huge represented integer does not imply huge state. This is the central reduction behind 2017 E Enigma; use the actual statement's leading-zero and output rules.

Multiplicative order

For gcd(a,m)=1, ord_m(a) is the smallest positive k with a^k=1 modulo m. Euler gives that it divides phi(m), not that it equals phi(m). Start k=phi(m), factor k, and repeatedly divide by prime p while a^(k/p)=1 modulo m.

Example ord_9(10)=1 although phi(9)=6. Our `order_mod` requires m>=2 and coprimality; it trial-factors m and phi(m), so factorization dominates for large numbers. With small bounded m and many queries, precompute factor tables instead. Generic cycle simulation works without coprimality but can have a tail and requires up to the number of states.

Worked regional reductions

Fuzzy Factorization: use the permitted error

The 2025 official statements allow an output integer Y satisfying |X-Y|/X<=10^-9, even though X can have 1001 decimal digits. Keep its first ten digits as m and let e be the number discarded. Then X=m*10^e+r, 0<=r<10^e, and m>=10^9. Choose Y=m*10^e. Therefore:

`|X-Y|/X = r/X < 1/m <= 10^-9`.

For at most ten digits, factor X exactly. Otherwise factor m<10^10 and add e to the exponents of 2 and 5. Merge repeated prime factors. This takes O(number of input digits + sqrt(m)) time; the large integer is never converted to a machine integer. The returned map from `fuzzy_factors` describes Y, not generally X. It assumes positive canonical decimal input. The empty map for X=1 represents an empty product; adapt the output to the statement's format.

Example X=123456789012345: m=1234567890, e=5. Since m=2*3^2*5*3607*3803, output factors of Y are 2^6*3^2*5^6*3607*3803. The discarded remainder is 12345. The exact check is `12345*10^9<=123456789012345`.

General rule: tolerance 10^-k can use k+1 leading significant digits to get the same bound, provided the resulting core can be factored cheaply. Check strictness, small inputs and the permitted prime bounds. This is a construction proved by an inequality; floating-point error estimates are unnecessary.

The earlier research checked this approach on all 109 released official F cases. The new C++ helper has separate local tests; it has not been submitted to the contest judge.

Displaying Decimals: separate one fraction from all pairs

For nonnegative a and positive b, reduce q=b/gcd(a,b). Write q=2^u*5^v*r with gcd(r,10)=1. There are max(u,v) nonrepeating fractional digits. If r=1, the expansion terminates; otherwise the repeating block has ord_r(10) digits. Thus the canonical display length, excluding point and overbar, is:

`digits10(floor(a/b)) + max(u,v) + (r==1 ? 0 : ord_r(10))`.

The representation stops once a terminating decimal ends; do not count arbitrary trailing zeros or the alternative recurring-9 representation. Integer zero counts as one digit. For a=0 reduction yields q=1, hence length one.

Fraction	Reduced denominator	Prefix length	Period	Integer digits	Total
1/4	4	2	0	1	3
1/6	6	1	1	1	3
3/14	14	1	6	1	8
41/4	4	2	0	2	4

Long division explains the period: after removing the 2/5 prefix, returning to the same remainder requires multiplication by 10^k to equal one modulo r. The coprime reduced numerator cancels. `decimal_shape` returns these three parts and uses trial division for order; do not call it N squared times at N=1e5.

For occurrence values bounded by V, aggregate the fractional length by exact gcd classes using the histogram section. Count integer digits separately: each quotient has at least one; add one for each threshold a>=10*b, a>=100*b, etc. Prefix frequencies answer each threshold count. Then divide the digit sum by N squared in the required modular field if the denominator is invertible.

For [15,36,14], the nine ordered lengths sum to 30; expectation is 30/9=10/3. Under modulus 998244353 this is 332748121. The earlier study compared the length formula with 10,000 direct long divisions and 18 small released D cases. Full aggregation runtime remains unverified; this book supplies the safe single-fraction building block, not a claimed full accepted D solution.

Counting without enumerating every object

Product rule, cases, combinations and recurrence

If each first choice has exactly B valid second choices, A first choices yield A*B outcomes. If the number depends on the first choice, sum those counts instead. Disjoint cases add. Overlapping cases require subtracting the intersection or assigning each outcome to exactly one case. This is a counting condition, not a statement about probabilistic independence.

For n distinct objects: ordered k-tuples without repetition number n!/(n-k)!; unordered k-subsets number C(n,k); ordered choices with repetition number n^k. Five people give 20 president/secretary assignments but 10 two-person committees. For identical items placed in k nonempty ordered bins with total n, stars and bars gives C(n-1,k-1); allowing empty bins gives C(n+k-1,k-1). Extra upper bounds require more reasoning.

Pascal's recurrence C(n,k)=C(n-1,k-1)+C(n-1,k), with edges one, works modulo any positive modulus using only addition. O(n^2) time; O(n) memory for one row updated in descending k. Descending order prevents using this row's updated values twice.

`PrimeCombinations(N,p)` precomputes factorials and inverse factorials in O(N+log p) time and O(N) memory, answers each query in O(1) time. It requires prime p and N<p. The constructor trusts primality. At N>=p, N! is zero modulo p and cannot be inverted. For a composite modulus, even N<p does not guarantee invertibility. Use Pascal for small n, exact cancellation for bounded exact results, or an advanced prime-power/CRT method. Lucas's theorem and prime-power combinations are outside this helper.

Original KACTL references: `content/combinatorial/factorial.tex`, `multinomial.h`. Never infer the correct combinatorial model from the existence of a factorial snippet.

Contribution counting and complements

An element at zero-based index i belongs to (i+1)*(n-i) contiguous subarrays: choose the left endpoint in 0..i and the right endpoint in i..n-1. Therefore the sum of every subarray sum is `sum(a[i]*(i+1)*(n-i))`, in O(n) time and O(1) extra memory. For [2,1,3], contributions are 6,4,9; total 19. Widen before multiplying all three terms.

For sums over unordered pairs, each value appears n-1 times, so sum(a[i]+a[j], i<j)=(n-1)*sum(a). For squared differences, expand algebra: sum over i<j of (a[i]-a[j])^2 equals n*sum(a[i]^2)-sum(a[i])^2. This can need more than 64 bits even if each value fits int.

For at least one desired feature, try total minus zero desired features. A linear binary string's all-bad substrings contribute L*(L+1)/2 for each maximal bad run of length L. Circular strings need a separate treatment of the wrapped run and the all-bad case; copying the linear formula blindly double counts or misses valid slices. These are the counting questions exposed by 2019 E Eggfruit Cake.

Inversions, duplicate values and repeated blocks

An inversion is a pair i<j with a[i]>a[j], strictly. Equal values do not count. Scan left to right and add seen-count(previous values <= current). A Fenwick tree over compressed values supplies that count. `inversions` implements this in O(n log n) time, O(n) memory, preserving the input; empty input returns zero. The maximum n*(n-1)/2 must fit its signed 64-bit result.

For K copies of the same sequence S, split inversions into within-copy and cross-copy pairs:

`total = K*within + K*(K-1)/2*cross`.

Here cross counts pairs of positions u,v in one copy with S[u]>S[v], regardless of their within-copy order, because every position of an earlier copy precedes every position of a later copy. If freq[x] is known, cross=sum(freq[x]*count(values<x)). Example S=ba has within=1,cross=1; three copies give 3+3=6. Do not use K*(K-1) instead of the unordered pair count of copies. Reduce products modulo the required modulus or use wide integers. This is the useful decomposition in 2023 I Inversions, official solutions.

Probability and expectations

Establish the sample space

Probability is favorable outcomes divided by all outcomes only when the elementary outcomes are equiprobable. Choosing a random pair of occurrences can differ from choosing a random pair of distinct values. Choosing two objects with replacement yields N squared ordered outcomes; without replacement yields N*(N-1). Equal values still represent separate occurrences if the experiment selects positions.

Expectation is sum(value*probability). It can be fractional and need not be a possible outcome: a uniform choice among 2,4,9 has expectation 5. For indicators I_j of events, E[sum I_j]=sum P(event_j). Independence is unnecessary for this addition. Multiplying P(A)*P(B) does need independence; generally P(A and B)=P(A)*P(B given A).

Example: a random permutation of n distinct values has one inversion for each unordered pair with probability 1/2. Expected inversions are n*(n-1)/4. Different pairs' indicators are dependent, but linearity still applies. Derive pair probabilities before introducing a DP over all permutations.

Modular fractions and process recurrences

To output A/B modulo prime p, compute A*inverse(B,p) only when B is not divisible by p. Reduce A,B as mathematical integers first if cancellation is required; a denominator zero modulo p cannot be inverted. A decimal approximation cannot be converted into the modular result. For 10/3 modulo 998244353, multiply 10 by inverse(3).

An expectation DP for a process must account for self loops. If E=cost+q*E+sum(p_j*E_j), then E=(cost+sum(p_j*E_j))/(1-q), provided q<1 over real probabilities. If q=1 and positive cost persists, expected cost is infinite. In modular arithmetic, even a real finite expression can have a noninvertible denominator modulo p; inspect the statement's guarantee.

Expectation is not interchangeable with a nonlinear function: E[X squared] generally differs from E[X] squared, and E[1/X] differs from 1/E[X]. Variance is E[X squared]-E[X] squared. When the process is complicated, classify contributions into a small number of symmetric types before coding. 2021 L Leaving Yharnam motivated that approach in our research; its independently derived formula passed the released tests, but it is not attributed to an official editorial.

Invariants, inequalities and choosing a proof

What must remain true?

Write an invariant as a precise equation or property, prove it initially, then prove one allowed operation preserves it. Swapping two entries preserves the multiset and sum. Adding one to one position and subtracting one from another preserves sum. Changing a value by a multiple of d preserves its residue modulo d. A move that flips two bits preserves parity of the number of ones. These facts give necessary conditions; they do not automatically prove reachability.

For sufficiency, construct a sequence of moves or prove an inductive reduction to a canonical state. If a move changes parity, a parity invariant is wrong even when small examples look persuasive. For impartial games, classify legal successor states before asserting a winning strategy; a numerical invariant alone may not decide whose turn matters.

When searching for an invariant, test sum, weighted sum, parity, gcd, order, multiset and connected components. If the input permits repeated operations, examine what one operation changes. Record any bound or monotonic quantity too: a strict decrease in a nonnegative integer proves termination.

Bounds, monotonicity and exchange arguments

A proposed answer is valid only if it satisfies every original inequality. For truncation, bound discarded magnitude relative to the original number; Fuzzy Factorization demonstrates this. For binary search on an answer, write a monotone predicate: if capacity C is feasible, every larger capacity must also be feasible. A fast check without monotonicity does not justify binary search.

For a greedy rule, try an exchange: take an optimal solution differing at the first choice, replace that choice with the greedy one, and show feasibility and objective do not worsen. If you cannot complete that argument, compare with exhaustive tiny cases. Small counterexamples are useful; passing small cases is still not a proof.

Useful algebra: arithmetic sum 1+...+n=n*(n+1)/2; geometric sum 1+r+...+r^k; triangle inequality |a+b|<=|a|+|b|; nonnegative square (a-b)^2>=0. For modular output, divide an even factor before multiplication or use an inverse only when valid. For floating-point comparisons, prefer an exact integer inequality when all inputs and thresholds can be scaled exactly.

Integer geometry essentials

Dot, cross, orientation and distance

For displacement vectors u,v, dot=u.x*v.x+u.y*v.y, cross=u.x*v.y-u.y*v.x. Dot zero means perpendicular; dot positive/negative means an acute/obtuse angle for nonzero vectors. Cross sign gives orientation. `orient(A,B,C)` is cross(B-A,C-A): positive left, negative right, zero collinear. A zero vector does not define an angle.

Squared distance is (dx)^2+(dy)^2. Compare squared distances when no actual length is requested; a square root adds rounding and does not change ordering. `dot`, `orient`, `dist2` use 128-bit arithmetic. Their documented coordinate magnitude bound is 1e18; cross/distance differences are then below signed 128-bit capacity. For arbitrary full-range `long long` coordinates, even signed 128 bits may overflow. Check the actual coordinate limit.

Example A=(0,0), B=(3,0), C=(1,2) gives orient=6, a left turn. The triangle area is |orient|/2 =3. It is half-integral for integer coordinates, so keep twice-area as an integer and print .0/.5 if required.

Segments and polygon area

Point P lies on closed segment AB exactly when orient(A,B,P)=0 and both coordinates lie within the respective inclusive endpoint ranges. This also handles A=B. Two noncollinear segments intersect when their endpoint orientations lie on opposite sides for both directed lines. Include zero cases by on-segment tests; for collinear segments, interval overlap decides. Compare signs rather than multiplying cross products: the product of two 128-bit cross products can overflow.

For polygon vertices in boundary order, signed twice-area is sum cross(P[i],P[(i+1)%n]). Absolute value gives twice the ordinary area of a simple polygon. Self-intersecting input instead gives algebraic signed area; sorting points by x does not produce the polygon boundary. For n vertices, accumulation may need a larger bound than an individual cross product. Translate all points by a nearby origin if that reduces magnitude, but still prove a bound.

Original references: `content/geometry/Point.h`, `SegmentIntersection.h`, `PolygonArea.h`. Integer predicates and floating-point constructions have different requirements: exact orientation does not make a computed intersection coordinate exact. When the statement asks for distances or intersections with real values, select a scale-aware tolerance based on its error guarantee; there is no universal EPS that repairs every algorithm.

Scope and evidence

The original KACTL remains the source for advanced algorithms: deterministic primality testing, Pollard factorization, modular roots/logarithms, fast transforms, convex hulls and other geometry. Use its source comments and tests to inspect dependencies before typing a snippet. This companion intentionally provides slower transparent arithmetic where the constraints permit it.

Regional source anchors: 2017 judge presentation, 2020 statements, 2023 official solutions, 2025 statements. Detailed independent checks and limits are recorded in the workspace `outputs/2025-math-case-studies.md` and `outputs/math-foundations-research.md`; they distinguish statement-derived work from jury solutions.

The companion tests compare small instances with exhaustive or direct arithmetic for division, congruences, factorization, decimal periods, combinations, inversions and geometry. These checks cover the supplied helpers, not every advanced topic mentioned in prose. Judge acceptance is not claimed. Before printing, confirm the event's paper and page rules and practice locating and adapting these entries in a mock contest.

Copyable C++17 helpers

The complete source below appears once. Its headers are standard library headers; `i128` itself is a GCC/Clang extension. Modules call earlier helpers, so retain those dependencies when extracting a section. Exceptions signal invalid domains or an unrepresentable result; in a contest submission, prove those cases impossible or handle them explicitly.

Code conventions and includes

```
#pragma once
#include <algorithm>
#include <cstdint>
#include <limits>
#include <map>
#include <numeric>
#include <optional>
#include <stdexcept>
#include <string>
#include <tuple>
#include <utility>
#include <vector>

namespace finals_math {
using i64 = long long;
using i128 = __int128_t; // GCC/Clang extension supported by our C++17 build.
```

Checked narrowing and signed division

Source: include/math.hpp | SHA256 fragment 79de2519

```
inline i64 narrow(i128 x) {
    if (x < std::numeric_limits<i64>::min() || x > std::numeric_limits<i64>::max())
        throw std::overflow_error("signed 64-bit result");
    return static_cast<i64>(x);
}
// b != 0; throws when the quotient cannot fit i64 (LLONG_MIN / -1).
inline i64 floor_div(i64 a, i64 b) {
    if (!b) throw std::invalid_argument("zero denominator");
    i128 x=a, y=b, q=x/y, r=x%y;
    if (r && ((r<0)!=(y<0))) --q;
    return narrow(q);
}
inline i64 ceil_div(i64 a, i64 b) {
    if (!b) throw std::invalid_argument("zero denominator");
    i128 x=a, y=b, q=x/y, r=x%y;
    if (r && ((r<0)==(y<0))) ++q;
    return narrow(q);
}
```

Exact decimals and digit counts

Source: include/math.hpp | SHA256 fragment 8cf28c55

```
// Exact fixed-point parse: optional sign; digits; optional dot and digits.
// Requires at least one digit before '.', <= places fractional digits, 0<=places<=18.
inline i64 parse_fixed(const std::string& s, int places) {
    if (places<0 || places>18) throw std::invalid_argument("decimal places");
    std::size_t j=0; bool neg=false;
    if (j<s.size() && (s[j]=='+' || s[j]=='-')) neg=s[j++]=='-';
    i128 value=0; int whole=0, frac=0;
    const i128 cap=i128(std::numeric_limits<i64>::max())+1;
    auto digit=[&](char c) {
        if (c<'0'||c>'9') throw std::invalid_argument("decimal digit");
        value=value*10+(c-'0');
        if (value>cap) throw std::overflow_error("decimal value");
    };
    while(j<s.size() && s[j]!='.') { digit(s[j++]); ++whole; }
    if(!whole) throw std::invalid_argument("integer part");
    if(j<s.size()) {
        ++j;
        while(j<s.size()) { digit(s[j++]); ++frac; }
    }
    if(frac>places) throw std::invalid_argument("too many fractional digits");
    while(frac++<places) {
        value*=10;
        if(value>cap) throw std::overflow_error("decimal value");
    }
    return narrow(neg ? -value : value);
}
inline int digits10(i64 x) { // Counts magnitude digits, including zero, excluding sign.
    i128 v=x; if(v<0) v=-v;
    int d=1; while(v>=10) { v/=10; ++d; } return d;
}
```

Modular arithmetic and binary powers

Source: include/math.hpp | SHA256 fragment 4e52c1ae

```
inline i64 norm(i64 a, i64 m) {
    if(m<=0) throw std::invalid_argument("positive modulus required");
    a%=m; return a<0 ? a+m : a;
}
inline i64 mul_mod(i64 a, i64 b, i64 m) {
    return i64(i128(norm(a,m))*norm(b,m)%m);
}
inline i64 pow_mod(i64 a, std::uint64_t e, i64 m) {
    a=norm(a,m); i64 r=1%m;
    while(e) { if(e&1) r=mul_mod(r,a,m); a=mul_mod(a,a,m); e>>=1; } return r;
}
```

Extended Euclid, inverses and CRT

Source: include/math.hpp | SHA256 fragment 7196125a

```
// Requires nonnegative a,b. Returns g,x,y with a*x+b*y=g.
inline std::tuple<i64,i128,i128> extended_gcd(i64 a,i64 b) {
    if(a<0||b<0) throw std::invalid_argument("nonnegative gcd inputs");
    i128 r=a,s=b,x=1,y=0,u=0,v=1;
    while(s) {
        i128 q=r/s;
        i128 t=r-q*s; r=s; s=t;
        t=x-q*u; x=u; u=t;
        t=y-q*v; y=v; v=t;
    }
    return {i64(r),x,y};
}
inline std::optional<i64> inverse(i64 a,i64 m) {
    if(m<=1) throw std::invalid_argument("inverse modulus must exceed one");
    auto [g,x,y]=extended_gcd(norm(a,m),m); (void)y;
    if(g!=1) return std::nullopt;
    return i64((x%m+m)%m);
}
struct Congruence { i64 residue, modulus; };
// Merge x=a (mod m), x=b (mod n); nullopt = incompatible.
// Positive moduli. Throws if the merged lcm exceeds LLONG_MAX.
inline std::optional<Congruence> crt(Congruence a,Congruence b) {
    a.residue=norm(a.residue,a.modulus); b.residue=norm(b.residue,b.modulus);
    auto [g,x,y]=extended_gcd(a.modulus,b.modulus); (void)y;
    i128 delta=i128(b.residue)-a.residue;
    if(delta%g) return std::nullopt;
    i64 n=b.modulus/g;
    i128 t=(delta/g*x)%n; if(t<0)t+=n;
    i64 lcm=narrow(i128(a.modulus)*n);
    i128 r=(i128(a.residue)+i128(a.modulus)*t)%lcm;
    return Congruence{i64(r),lcm};
}
```

Trial factorization, totients and sieve

Source: include/math.hpp | SHA256 fragment bcfce104

```
// Positive n; O(sqrt(n)) worst case. Suitable for n<=1e12 in modest counts.
inline std::vector<std::pair<i64,int>> factor(i64 n) {
    if(n<1) throw std::invalid_argument("positive factor input");
    std::vector<std::pair<i64,int>> f;
    for(i64 p=2;p<=n/p;p+=(p==2?1:2)) if(n%p==0) {
        int e=0; do {n/=p;++e;}while(n%p==0); f.push_back({p,e});
    }
    if(n>1)f.push_back({n,1}); return f;
}
inline i64 phi(i64 n) {
    i64 ans=n; for(auto [p,e]:factor(n)) { (void)e; ans=ans/p*(p-1); } return ans;
}
struct Sieve {
    std::vector<int> spf, primes;
    explicit Sieve(int n) {
        if(n<0) throw std::invalid_argument("nonnegative sieve limit");
        spf.resize(std::size_t(n)+1,0);
        for(int i=2;i<=n;++i) {
            if(!spf[i]) { spf[i]=i; primes.push_back(i); }
            for(int p:primes) { if(p>spf[i] || p>n/i)break; spf[p*i]=p; }
        }
    }
    std::vector<std::pair<i64,int>> factors(int x) const {
        if(x<1 || x>=int(spf.size())) throw std::invalid_argument("sieve range");
        std::vector<std::pair<i64,int>> f;
        while(x>1) {int p=spf[x],e=0;do{x/=p;++e;}while(x>1&&spf[x]==p);f.push_back({p,e});}
        return f;
    }
};
```

Multiplicative order and decimal shape

Source: include/math.hpp | SHA256 fragment 24fff7a6

```
inline i64 order_mod(i64 a,i64 m) { // m>=2, gcd(a,m)=1; trial division cost applies.
    if(m<2 || std::gcd(norm(a,m),m)!=1) throw std::invalid_argument("order requires a unit");
    i64 k=phi(m);
    for(auto [p,e]:factor(k)) { (void)e; while(k%p==0&&pow_mod(a,k/p,m)==1)k/=p; }
    return k;
}
struct DecimalShape { int integer_digits, nonrepeating; i64 period; };
// a>=0, b>=1; period computation may be costly.
inline DecimalShape decimal_shape(i64 a,i64 b) {
    if(a<0||b<1) throw std::invalid_argument("nonnegative fraction");
    i64 q=b/std::gcd(a,b); int u=0,v=0;
    while(q%2==0) {q/=2;++u;} while(q%5==0) {q/=5;++v;}
    return {digits10(a/b),std::max(u,v),q==1?0:order_mod(10,q)};
}
```

Combinations under a prime modulus

Source: include/math.hpp | SHA256 fragment 60d37354

```
// Prime p is a CALLER precondition; n<p. Constructor does not test primality.
struct PrimeCombinations {
    i64 p; std::vector<i64> fac,invfac;
    PrimeCombinations(int n,i64 prime):p(prime) {
        if(n<0||p<=1||n>=p)throw std::invalid_argument("need 0<=n<prime");
        fac.resize(n+1,1); invfac.resize(n+1,1);
        for(int i=1;i<=n;++i)fac[i]=mul_mod(fac[i-1],i,p);
        invfac[n]=pow_mod(fac[n],p-2,p);
        for(int i=n;i>0;--i)invfac[i-1]=mul_mod(invfac[i],i,p);
    }
    i64 choose(int n,int k)const {
        if(n<0||n>=int(fac.size()))throw std::invalid_argument("choose range");
        if(k<0||k>n)return 0;
        return mul_mod(fac[n],mul_mod(invfac[k],invfac[n-k],p),p);
    }
};
```

Strict inversions by compressed Fenwick tree

Source: include/math.hpp | SHA256 fragment c34e8578

```
// Generic comparisons: counts i<j with a[i]>a[j], O(n log n), preserves input.
inline i64 inversions(std::vector<i64> a) {
    auto vals=a;
    std::sort(vals.begin(),vals.end());
    vals.erase(std::unique(vals.begin(),vals.end()),vals.end());
    std::vector<i64> bit(vals.size()+1); i64 answer=0,seen=0;
    for(i64 x:a) {
        int k=int(std::lower_bound(vals.begin(),vals.end(),x)-vals.begin())+1;
        i64 le=0; for(int j=k;j;j-=j&-j)le+=bit[j];
        answer=narrow(i128(answer)+seen-le); ++seen;
        for(int j=k;j<int(bit.size());j+=j&-j)++bit[j];
    }
    return answer;
}
```

Fuzzy factorization construction

Source: include/math.hpp | SHA256 fragment 8f8ce765

```
// Positive canonical decimal input (no leading zero); factor the ten-digit prefix,
// then account for trailing powers of ten. Empty factors means Y=1.
inline std::map<i64,int> fuzzy_factors(const std::string& s) {
    if(s.empty()||s[0]=='0')throw std::invalid_argument("positive canonical decimal");
    for(char c:s)if(c<'0'||c>'9')throw std::invalid_argument("decimal digits");
    i64 m=0; int keep=int(std::min<std::size_t>(10,s.size()));
    for(int i=0;i<keep;++i)m=m*10+s[i]-'0';
    int e=int(s.size())-keep; std::map<i64,int> f;
    for(auto [p,k]:factor(m))f[p]=k;
    if(e) {f[2]+=e;f[5]+=e;} return f;
}
```

Exact integer geometry

Source: include/math.hpp | SHA256 fragment b647118f

```
struct Point { i64 x,y; };
// Coordinate magnitudes <=1e18; products/sums fit signed 128 bits under this bound.
inline i128 orient(Point a,Point b,Point c) {
    return (i128(b.x)-a.x)*(i128(c.y)-a.y)-(i128(b.y)-a.y)*(i128(c.x)-a.x);
}
inline i128 dot(Point a,Point b) {return i128(a.x)*b.x+i128(a.y)*b.y;}
inline i128 dist2(Point a,Point b) {i128 x=i128(a.x)-b.x,y=i128(a.y)-b.y;return x*x+y*y;}
} // namespace finals_math
```